

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

A - Question #2

Let's consider a generic MIPS processor that executes 32-bit instructions, uses 32-bit addresses, and performs branch predictions via a Branch History Table (BHT) composed of 32 entries.

Assuming that the BHT is initially empty (i.e., full of 0s), report the content of the BHT during the execution of the following program:

ADDR	CODE
	.data
	vec: .word 2, 8, 4, 2, 8, 2, 4 # input vector
	res1: .space 8 # number of values equal to the 1 st element
	res2: .space 8 # number of values equal to the 2 nd element
	res3: .space 8 # number of values equal to the 3 rd element
	.text
0x0000	daddui r1, r0, 4 # initialize the 1 st element to be compared
0x0004	daddui r2, r0, 8 # initialize the 2 nd value to be compared
0x0008	daddui r3, r0, 16 # initialize the 3 rd value to be compared
0x000c	daddui r4, r0, 0 # number of values equal to the 1 st element
0x0010	daddui r5, r0, 0 # number of values equal to the 2 nd element
0x0014	daddui r6, r0, 0 # number of values equal to the 3 rd element
0x0018	daddui r7, r0, -1 # initialize the loops counter
0x001c	daddui r8, r0, 7 # condition for exiting the cycle
0x0020	daddui r9, r0, -8 # initialize the pointer
0x0024	lop: daddui r7, r7, 1 # increment the loops counter
0x0028	beq r7, r8, trm # condition for exiting the cycle
0x002c	daddui r9, r9, 8 # increment the pointer
0x0030	ld r10, vec(r9) # load a value from vec
0x0034	dsll r10, r10, 1 # perform a 1-bit shift left
0x0038	bne r10, r1, lb1 # check if the value is equal to the 1 st element
0x003c	daddui r4, r4, 1 # increment the counter of values equal to the 1 st element
0x0040	j lop # next cycle
0x0044	lb1: bne r10, r2, lb2 # check if the value is equal to the 2 nd element
0x0048	daddui r5, r5, 1 # increment the counter of values equal to the 2 nd element
0x004c	j lop # next cycle
0x0050	lb2: bne r10, r3, lop # check if the value is equal to the 3 rd element
0x0054	daddui r6, r6, 1 # increment the counter of values equal to the 3 rd element
0x0058	j lop # next cycle
0x005c	trm: sd r4, res1(r0) # store the number of values equal to the 1 st element
0x0060	sd r5, res2(r0) # store the number of values equal to the 2 nd element
0x0064	sd r6, res3(r0) # store the number of values equal to the 3 rd element
0x0068	halt # termination of the program

a) Try to understand what the code does using the code itself and the comments provided.

Explanation:

The given code takes the values in vec, performs a power of 2 on them, and checks whether they are equal to 4, 8, or 16. If they are equal to one of these numbers, the corresponding counter is incremented. After execution, the counters are stored in the variables r1, r2, and r3.

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

- b) Fill in the contents of this table to understand which BHT entry the instructions of interest will refer to. Add more lines if needed.

Instruction	Address (Hex)	Address (Binary)	Entry #
beq r7, r8, trm	0x0028	0000 0000 0000 0000 0000 0000 0010 1000	10
bne r10, r1, lb1	0x0038	0000 0000 0000 0000 0000 0000 0011 1000	14
bne r10, r2, lb2	0x0044	0000 0000 0000 0000 0000 0000 0100 0100	17
bne r10, r3, lop	0x0050	0000 0000 0000 0000 0000 0000 0101 0000	20

- c) Run the program and describe the change of BHT during its execution. Write in the right cells whether the prediction of the current branch (pred) and the actual behavior (result) of the software is Taken (T) or Not Taken (NT). Use as many iterations you think are necessary and, if the table is not sufficient to describe them, duplicate it to describe the entire execution of the program.

ADDR	Code	BHT entry #	Iteration #1		Iteration #2		Iteration #3		Iteration #4	
			pred	res	pred	res	pred	res	pred	res
0x0000	daddui r1, r0, 4									
0x0004	daddui r2, r0, 8									
0x0008	daddui r3, r0, 16									
0x000c	daddui r4, r0, 0									
0x0010	daddui r5, r0, 0									
0x0014	daddui r6, r0, 0									
0x0018	daddui r7, r0, -1									
0x001c	daddui r8, r0, 7									
0x0020	daddui r9, r0, -8									
0x0024	lop: daddui r7, r7, 1									
0x0028	beq r7, r8, trm	10	NT	NT	NT	NT	NT	NT	NT	NT
0x002c	daddui r9, r9, 8									
0x0030	ld r10,v1(r9)									
0x0034	dsll r10, r10, 1									
0x0038	bne r10, r1, lb1	14	NT	NT	NT	T	T	T	T	NT
0x003c	daddui r4, r4, 1									
0x0040	j lop									
0x0044	lb1: bne r10, r2, lb2	17			NT	T	T	NT		
0x0048	daddui r5, r5, 1									
0x004c	j lop									
0x0050	lb2: bne r10, r3, lop	20			NT	NT				
0x0054	daddui r6, r6, 1									
0x0058	j lop									
0x005c	trm: sd r4, res1(r0)									
0x0060	sd r5, res2(r0)									
0x0064	sd r6, res3(r0)									
0x0068	halt									

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

ADDR	Code	BHT	Iteration #5		Iteration #6		Iteration #7		Iteration #8	
		entry #	pred	res	pred	res	pred	res	pred	res
0x0000	daddui r1, r0, 4									
0x0004	daddui r2, r0, 8									
0x0008	daddui r3, r0, 16									
0x000c	daddui r4, r0, 0									
0x0010	daddui r5, r0, 0									
0x0014	daddui r6, r0, 0									
0x0018	daddui r7, r0, -1									
0x001c	daddui r8, r0, 7									
0x0020	daddui r9, r0, -8									
0x0024	lop: daddui r7, r7, 1									
0x0028	beq r7, r8, trm	10	NT	NT	NT	NT	NT	NT	NT	T
0x002c	daddui r9, r9, 8									
0x0030	ld r10,v1(r9)									
0x0034	dsll r10, r10, 1									
0x0038	bne r10, r1, lb1	14	NT	T	T	NT	NT	T		
0x003c	daddui r4, r4, 1									
0x0040	j lop									
0x0044	lb1: bne r10, r2, lb2	17	NT	T			T	NT		
0x0048	daddui r5, r5, 1									
0x004c	j lop									
0x0050	lb2: bne r10, r3, lop	20	NT	NT						
0x0054	daddui r6, r6, 1									
0x0058	j lop									
0x005c	trm: sd r4, res1(r0)									
0x0060	sd r5, res2(r0)									
0x0064	sd r6, res3(r0)									
0x0068	halt									

- d) Write the total number of correct and incorrect predictions and report the BHT final content on the designated table.

Correctly predicted branches: 11

Mispredicted branches: 10

BHT - Final content

Entry 0	0	Entry 8	0	Entry 16	0	Entry 24	0
Entry 1	0	Entry 9	0	Entry 17	0	Entry 25	0
Entry 2	0	Entry 10	1	Entry 18	0	Entry 26	0
Entry 3	0	Entry 11	0	Entry 19	0	Entry 27	0
Entry 4	0	Entry 12	0	Entry 20	0	Entry 28	0
Entry 5	0	Entry 13	0	Entry 21	0	Entry 29	0
Entry 6	0	Entry 14	1	Entry 22	0	Entry 30	0
Entry 7	0	Entry 15	0	Entry 23	0	Entry 31	0

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

B - Question #2

Let's consider a generic MIPS processor that executes 32-bit instructions, uses 32-bit addresses, and performs branch predictions via a Branch History Table (BHT) composed of 32 entries. Assuming that the BHT is initially empty (i.e., full of 0s), report the content of the BHT during the execution of the following program:

ADDR	CODE	
	.data	
	vec: .word 32, 16, 32, 8, 16,	# input vector
	16, 8, 8	
	res1: .space 8	# number of values equal to the 1 st element
	res2: .space 8	# number of values equal to the 2 nd element
	res3: .space 8	# number of values equal to the 3 rd element
	.text	
0x0034	daddui r1, r0, 32	# initialize the 1 st element to be compared
0x0038	daddui r2, r0, 64	# initialize the 2 nd value to be compared
0x003c	daddui r3, r0, 128	# initialize the 3 rd value to be compared
0x0040	daddui r4, r0, 0	# number of values equal to the 1 st element
0x0044	daddui r5, r0, 0	# number of values equal to the 2 nd element
0x0048	daddui r6, r0, 0	# number of values equal to the 3 rd element
0x004c	daddui r7, r0, -1	# initialize the loops counter
0x0050	daddui r8, r0, 8	# condition for exiting the cycle
0x0054	daddui r9, r0, -8	# initialize the pointer
0x0058	lop: daddui r7, r7, 1	# increment the loops counter
0x005c	beq r7, r8, trm	# condition for exiting the cycle
0x0060	daddui r9, r9, 8	# increment the pointer
0x0064	ld r10, vec(r9)	# load a value from vec
0x0068	dsll r10, r10, 2	# perform a 2-bit shift left
0x006c	bne r10, r1, lb1	# check if the value is equal to the 1 st element
0x0070	daddui r4, r4, 1	# increment the counter of values equal to the 1 st element
0x0074	j lop	# next cycle
0x0078	lb1: bne r10, r2, lb2	# check if the value is equal to the 2 nd element
0x007c	daddui r5, r5, 1	# increment the counter of values equal to the 2 nd element
0x0080	j lop	# next cycle
0x0084	lb2: bne r10, r3, lop	# check if the value is equal to the 3 rd element
0x0088	daddui r6, r6, 1	# increment the counter of values equal to the 3 rd element
0x008c	j lop	# next cycle
0x0090	trm: sd r4, res1(r0)	# store the number of values equal to the 1 st element
0x0094	sd r5, res2(r0)	# store the number of values equal to the 2 nd element
0x0098	sd r6, res3(r0)	# store the number of values equal to the 3 rd element
0x009c	halt	# termination of the program

e) Try to understand what the code does using the code itself and the comments provided.

Explanation:

The given code takes the values in vec, performs a power of 2 on them twice, and checks whether they are equal to 32, 64, or 128. If they are equal to one of these numbers, the corresponding counter is incremented. After execution, the counters are stored in the variables r1, r2, and r3.

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

- f) Fill in the contents of this table to understand which BHT entry the instructions of interest will refer to. Add more lines if needed.

Instruction	Address (Hex)	Address (Binary)	Entry #
beq r7, r8, trm	0x005c	0000 0000 0000 0000 0000 0000 0101 1100	23
bne r10, r1, lb1	0x006c	0000 0000 0000 0000 0000 0000 0110 1100	27
bne r10, r2, lb2	0x0078	0000 0000 0000 0000 0000 0000 0111 1000	30
bne r10, r3, lop	0x0084	0000 0000 0000 0000 0000 0000 1000 0100	1

- g) Run the program and describe the change of BHT during its execution. Write in the right cells whether the prediction of the current branch (pred) and the actual behavior (result) of the software is Taken (T) or Not Taken (NT). Use as many iterations you think are necessary and, if the table is not sufficient to describe them, duplicate it to describe the entire execution of the program.

ADDR	Code	BHT	Iteration #1		Iteration #2		Iteration #3		Iteration #4	
		entry #	pred	res	pred	res	pred	res	pred	res
0x0034	daddui r1, r0, 32									
0x0038	daddui r2, r0, 64									
0x003c	daddui r3, r0, 128									
0x0040	daddui r4, r0, 0									
0x0044	daddui r5, r0, 0									
0x0048	daddui r6, r0, 0									
0x004c	daddui r7, r0, -1									
0x0050	daddui r8, r0, 8									
0x0054	daddui r9, r0, -8									
0x0058	lop: daddui r7, r7, 1									
0x005c	beq r7, r8, trm	23	NT	NT	NT	NT	NT	NT	NT	NT
0x0060	daddui r9, r9, 8									
0x0064	ld r10, v1(r9)									
0x0068	dsll r10, r10, 2									
0x006c	bne r10, r1, lb1	27	NT	T	T	T	T	T	T	NT
0x0070	daddui r4, r4, 1									
0x0074	j lop									
0x0078	lb1: bne r10, r2, lb2	30	NT	T	T	NT	NT	T		
0x007c	daddui r5, r5, 1									
0x0080	j lop									
0x0084	lb2: bne r10, r3, lop	1	NT	NT			NT	NT		
0x0088	daddui r6, r6, 1									
0x008c	j lop									
0x0090	trm: sd r4, res1(r0)									
0x0094	sd r5, res2(r0)									
0x0098	sd r6, res3(r0)									
0x009c	halt									

ADDR	Code	BHT	Iteration #5		Iteration #6		Iteration #7		Iteration #8	
		entry #	pred	res	pred	res	pred	res	pred	res
0x0034	daddui r1, r0, 32									
0x0038	daddui r2, r0, 64									
0x003c	daddui r3, r0, 128									
0x0040	daddui r4, r0, 0									
0x0044	daddui r5, r0, 0									
0x0048	daddui r6, r0, 0									
0x004c	daddui r7, r0, -1									
0x0050	daddui r8, r0, 8									
0x0054	daddui r9, r0, -8									
0x0058	lop: daddui r7, r7, 1									

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

0x005c	beq r7, r8, trm	23	NT	NT	NT	NT	NT	NT	NT	NT
0x0060	daddui r9, r9, 8									
0x0064	ld r10,v1(r9)									
0x0068	dsll r10, r10, 2									
0x006c	bne r10, r1, lb1	27	NT	T	T	T	T	NT	NT	NT
0x0070	daddui r4, r4, 1									
0x0074	j lop									
0x0078	lb1: bne r10, r2, lb2	30	T	NT	NT	NT				
0x007c	daddui r5, r5, 1									
0x0080	j lop									
0x0084	lb2: bne r10, r3, lop	1								
0x0088	daddui r6, r6, 1									
0x008c	j lop									
0x0090	trm: sd r4, res1(r0)									
0x0094	sd r5, res2(r0)									
0x0098	sd r6, res3(r0)									
0x009c	halt									

ADDR	Code	BHT entry #	Iteration #9		Iteration #10		Iteration #11		Iteration #12	
			pred	res	pred	res	pred	res	pred	res
0x0034	daddui r1, r0, 32									
0x0038	daddui r2, r0, 64									
0x003c	daddui r3, r0, 128									
0x0040	daddui r4, r0, 0									
0x0044	daddui r5, r0, 0									
0x0048	daddui r6, r0, 0									
0x004c	daddui r7, r0, -1									
0x0050	daddui r8, r0, 8									
0x0054	daddui r9, r0, -8									
0x0058	lop: daddui r7, r7, 1									
0x005c	beq r7, r8, trm	23	NT	T						
0x0060	daddui r9, r9, 8									
0x0064	ld r10,v1(r9)									
0x0068	dsll r10, r10, 2									
0x006c	bne r10, r1, lb1	27								
0x0070	daddui r4, r4, 1									
0x0074	j lop									
0x0078	lb1: bne r10, r2, lb2	30								
0x007c	daddui r5, r5, 1									
0x0080	j lop									
0x0084	lb2: bne r10, r3, lop	1								
0x0088	daddui r6, r6, 1									
0x008c	j lop									
0x0090	trm: sd r4, res1(r0)									
0x0094	sd r5, res2(r0)									
0x0098	sd r6, res3(r0)									
0x009c	halt									

h) Write the total number of correct and incorrect predictions and report the BHT final content on the designated table.

Correctly predicted branches: 15

Mispredicted branches: 9

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

BHT - Final content

Entry 0	0	Entry 8	0	Entry 16	0	Entry 24	0
Entry 1	0	Entry 9	0	Entry 17	0	Entry 25	0
Entry 2	0	Entry 10	0	Entry 18	0	Entry 26	0
Entry 3	0	Entry 11	0	Entry 19	0	Entry 27	0
Entry 4	0	Entry 12	0	Entry 20	0	Entry 28	0
Entry 5	0	Entry 13	0	Entry 21	0	Entry 29	0
Entry 6	0	Entry 14	0	Entry 22	0	Entry 30	0
Entry 7	0	Entry 15	0	Entry 23	1	Entry 31	0

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

B - Question #2, alternative solution with 1-bit shifts

Let's consider a generic MIPS processor that executes 32-bit instructions, uses 32-bit addresses, and performs branch predictions via a Branch History Table (BHT) composed of 32 entries.

Assuming that the BHT is initially empty (i.e., full of 0s), report the content of the BHT during the execution of the following program:

ADDR	CODE
	.data
	vec: .word 32, 16, 32, 8, 16, 16, 8, 8 # input vector
	res1: .space 8 # number of values equal to the 1 st element
	res2: .space 8 # number of values equal to the 2 nd element
	res3: .space 8 # number of values equal to the 3 rd element
	.text
0x0034	daddui r1, r0, 32 # initialize the 1 st element to be compared
0x0038	daddui r2, r0, 64 # initialize the 2 nd value to be compared
0x003c	daddui r3, r0, 128 # initialize the 3 rd value to be compared
0x0040	daddui r4, r0, 0 # number of values equal to the 1 st element
0x0044	daddui r5, r0, 0 # number of values equal to the 2 nd element
0x0048	daddui r6, r0, 0 # number of values equal to the 3 rd element
0x004c	daddui r7, r0, -1 # initialize the loops counter
0x0050	daddui r8, r0, 8 # condition for exiting the cycle
0x0054	daddui r9, r0, -8 # initialize the pointer
0x0058	lop: daddui r7, r7, 1 # increment the loops counter
0x005c	beq r7, r8, trm # condition for exiting the cycle
0x0060	daddui r9, r9, 8 # increment the pointer
0x0064	ld r10, vec(r9) # load a value from vec
0x0068	dsll r10, r10, 1 # perform a 1-bit shift left
0x006c	bne r10, r1, lb1 # check if the value is equal to the 1 st element
0x0070	daddui r4, r4, 1 # increment the counter of values equal to the 1 st element
0x0074	j lop # next cycle
0x0078	lb1: bne r10, r2, lb2 # check if the value is equal to the 2 nd element
0x007c	daddui r5, r5, 1 # increment the counter of values equal to the 2 nd element
0x0080	j lop # next cycle
0x0084	lb2: bne r10, r3, lop # check if the value is equal to the 3 rd element
0x0088	daddui r6, r6, 1 # increment the counter of values equal to the 3 rd element
0x008c	j lop # next cycle
0x0090	trm: sd r4, res1(r0) # store the number of values equal to the 1 st element
0x0094	sd r5, res2(r0) # store the number of values equal to the 2 nd element
0x0098	sd r6, res3(r0) # store the number of values equal to the 3 rd element
0x009c	halt # termination of the program

i) Try to understand what the code does using the code itself and the comments provided.

Explanation:

The given code takes the values in *vec*, performs a power of 2 on them, and checks whether they are equal to 32, 64, or 128. If they are equal to one of these numbers, the corresponding counter is incremented. After execution, the counters are stored in the variables *r1*, *r2*, and *r3*.

Note that, in this specific case, no value in *vec* will ever be equal to 128: consequently, the result of the branch instruction *bne r10, r2, lb2*, after the first execution, will always be T.

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

- j) Fill in the contents of this table to understand which BHT entry the instructions of interest will refer to. Add more lines if needed.

Instruction	Address (Hex)	Address (Binary)	Entry #
beq r7, r8, trm	0x005c	0000 0000 0000 0000 0000 0000 0101 1100	23
bne r10, r1, lb1	0x006c	0000 0000 0000 0000 0000 0000 0110 1100	27
bne r10, r2, lb2	0x0078	0000 0000 0000 0000 0000 0000 0111 1000	30
bne r10, r3, lop	0x0084	0000 0000 0000 0000 0000 0000 1000 0100	1

- k) Run the program and describe the change of BHT during its execution. Write in the right cells whether the prediction of the current branch (pred) and the actual behavior (result) of the software is Taken (T) or Not Taken (NT). Use as many iterations you think are necessary and, if the table is not sufficient to describe them, duplicate it to describe the entire execution of the program.

ADDR	Code	BHT	Iteration #1		Iteration #2		Iteration #3		Iteration #4	
		entry #	pred	res	pred	res	pred	res	pred	res
0x0034	daddui r1, r0, 32									
0x0038	daddui r2, r0, 64									
0x003c	daddui r3, r0, 128									
0x0040	daddui r4, r0, 0									
0x0044	daddui r5, r0, 0									
0x0048	daddui r6, r0, 0									
0x004c	daddui r7, r0, -1									
0x0050	daddui r8, r0, 8									
0x0054	daddui r9, r0, -8									
0x0058	lop: daddui r7, r7, 1									
0x005c	beq r7, r8, trm	23	NT	NT	NT	NT	NT	NT	NT	NT
0x0060	daddui r9, r9, 8									
0x0064	ld r10, v1(r9)									
0x0068	dsll r10, r10, 2									
0x006c	bne r10, r1, lb1	27	NT	T	T	NT	NT	T	T	T
0x0070	daddui r4, r4, 1									
0x0074	j lop									
0x0078	lb1: bne r10, r2, lb2	30	NT	NT			NT	NT	NT	T
0x007c	daddui r5, r5, 1									
0x0080	j lop									
0x0084	lb2: bne r10, r3, lop	1							NT	T
0x0088	daddui r6, r6, 1									
0x008c	j lop									
0x0090	trm: sd r4, res1(r0)									
0x0094	sd r5, res2(r0)									
0x0098	sd r6, res3(r0)									
0x009c	halt									

ADDR	Code	BHT	Iteration #5		Iteration #6		Iteration #7		Iteration #8	
		entry #	pred	res	pred	res	pred	res	pred	res
0x0034	daddui r1, r0, 32									
0x0038	daddui r2, r0, 64									
0x003c	daddui r3, r0, 128									
0x0040	daddui r4, r0, 0									
0x0044	daddui r5, r0, 0									
0x0048	daddui r6, r0, 0									
0x004c	daddui r7, r0, -1									
0x0050	daddui r8, r0, 8									
0x0054	daddui r9, r0, -8									
0x0058	lop: daddui r7, r7, 1									

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

0x005c	beq r7, r8, trm	23	NT	NT	NT	NT	NT	NT	NT	NT
0x0060	daddui r9, r9, 8									
0x0064	ld r10,v1(r9)									
0x0068	dsll r10, r10, 2									
0x006c	bne r10, r1, lb1	27	T	NT	NT	NT	NT	T	T	T
0x0070	daddui r4, r4, 1									
0x0074	j lop									
0x0078	lb1: bne r10, r2, lb2	30					T	T	T	T
0x007c	daddui r5, r5, 1									
0x0080	j lop									
0x0084	lb2: bne r10, r3, lop	1					T	T	T	T
0x0088	daddui r6, r6, 1									
0x008c	j lop									
0x0090	trm: sd r4, res1(r0)									
0x0094	sd r5, res2(r0)									
0x0098	sd r6, res3(r0)									
0x009c	halt									

ADDR	Code	BHT entry #	Iteration #9		Iteration #10		Iteration #11		Iteration #12	
			pred	res	pred	res	pred	res	pred	res
0x0034	daddui r1, r0, 32									
0x0038	daddui r2, r0, 64									
0x003c	daddui r3, r0, 128									
0x0040	daddui r4, r0, 0									
0x0044	daddui r5, r0, 0									
0x0048	daddui r6, r0, 0									
0x004c	daddui r7, r0, -1									
0x0050	daddui r8, r0, 8									
0x0054	daddui r9, r0, -8									
0x0058	lop: daddui r7, r7, 1									
0x005c	beq r7, r8, trm	23	NT	T						
0x0060	daddui r9, r9, 8									
0x0064	ld r10,v1(r9)									
0x0068	dsll r10, r10, 2									
0x006c	bne r10, r1, lb1	27								
0x0070	daddui r4, r4, 1									
0x0074	j lop									
0x0078	lb1: bne r10, r2, lb2	30								
0x007c	daddui r5, r5, 1									
0x0080	j lop									
0x0084	lb2: bne r10, r3, lop	1								
0x0088	daddui r6, r6, 1									
0x008c	j lop									
0x0090	trm: sd r4, res1(r0)									
0x0094	sd r5, res2(r0)									
0x0098	sd r6, res3(r0)									
0x009c	halt									

1) Write the total number of correct and incorrect predictions and report the BHT final content on the designated table.

Correctly predicted branches: 17

Mispredicted branches: 8

Computer Architectures

Exam of 1.7.2025 - part I

First name, Last name, ID.....

BHT - Final content

Entry 0	0	Entry 8	0	Entry 16	0	Entry 24	0
Entry 1	1	Entry 9	0	Entry 17	0	Entry 25	0
Entry 2	0	Entry 10	0	Entry 18	0	Entry 26	0
Entry 3	0	Entry 11	0	Entry 19	0	Entry 27	1
Entry 4	0	Entry 12	0	Entry 20	0	Entry 28	0
Entry 5	0	Entry 13	0	Entry 21	0	Entry 29	0
Entry 6	0	Entry 14	0	Entry 22	0	Entry 30	1
Entry 7	0	Entry 15	0	Entry 23	1	Entry 31	0